

08/10/2020 - Sobre la practica

1. Duración : 1 mes.
2. Evalua: Manejo del API de procesos.
3. Sobre la practica.

Sobre la practica

- * 2 partes
 - Refuerzo
 - Shell sencillo (wish).

<https://github.com/dannymrock/SO-Lab2-20201>

▶ Ejercicios de refuerzo (punto 7)

- 7. Escriba un programa en C llamado **time.c** que determine la cantidad de tiempo necesaria para correr un comando desde la linea de comandos. Este programa será ejecutado como " **time <command>** " y mostrará la cantidad de tiempo gastada para ejecutar el comando especificado. Para resolver el problema haga uso de `fork()` y `exec()` , así como de la función `gettimeofday()` para determinar el tiempo transcurrido.

La estrategia general es hacer un fork para crear un proceso hijo el cual ejecutara el comando especificado. Sin embargo, antes de que el proceso hijo ejecute el comando especificado, deba almacenar el tiempo actual (**starting time**). El padre invocará el wait para esperar por la culminación del proceso hijo. Luego, una vez que el proceso hijo culmine, el padre almacenara el tiempo actual en este punto (**ending time**). La diferencia entre los tiempos **inicial** y **final** (**starting y endind**) representará el tiempo gastado para ejecutar el comando. Por ejemplo la salida en pantalla de abajo muestra la cantidad de tiempo para correr el comando `ls` :

```
./time ls
time.c
time

Elapsed time: 0.25422
```

▶ Enunciado (Hacer).

Enunciado - Unix Shell

Nota: Esta practica es una traducción de la práctica [Process Shell](#) del libro de Remzi. Esta traducción puede tener muchos errores y a veces no ser fiel con lo que el autor quiere transmitir. Si desea leerla en ingles puede encontrar esto en el siguiente [enlace](#).

En esta practica, usted construirá un simple Unix shell. El shell es el corazon de la interface de línea de comandos, y por lo tanto es el ambiente de programación central de Unix/C.

Es necesario dominar el uso del shell para volverse competitivo en este mundo; saber cómo se construye el shell en sí es el foco de este proyecto.

Que necesito saber?

API de procesos:

<https://github.com/dannymrock/UdeA-SO-Lab/tree/master/lab1>

1. getpid •
2. fork •
3. exec •
4. wait •

} Ejemplos

Diagram illustrating the process of obtaining the PID (Process ID) for a process:

The diagram shows a process **A** (represented by a circle with a spiral) pointing to a **PCB** (Process Control Block, represented by a rectangle). A red arrow points from the **PCB** to the text **pid (cedula)** with a '#' symbol. A blue arrow points from the text to the function **Funcion getpid()**.

La siguiente tabla detalla mas a fondo esta función:

2. fork (Obtiene una copia (o clon) del process padre)

↓
Llama a fork.

```
graph TD
    P((P)) -- "fork" --> H((H))
    P -- "fork" --> P2[ ]
    P2 -.-> P3[ ]
    H -- "fork" --> H2[ ]
    H2 -.-> H3[ ]
    P3 -.-> P4[ ]
    H3 -.-> H4[ ]
    P4 -.-> P5[ ]
    H4 -.-> H5[ ]
```

Diagram illustrating the fork system call:

- Parent process (P) with PID pid_p calls `fork`.
- The `fork` system call creates a child process (H) with PID pid_h .
- The return value of `fork` is 0 for the child process (H) and $\neq 0 (pid_p)$ for the parent process (P).
- Both parent and child processes continue their execution.

Función	<code>fork</code>
Sintaxis	<code>pid_t fork(void);</code>
Librerías necesarias	<code>#include <unistd.h></code>
Descripción	La función <code>fork()</code> crea un nuevo proceso hijo como duplicado del proceso que la invoca (padre).
Parámetros de entrada	No tiene
Valor retornado	Cuando la ejecución de la función es exitosa se retorna un valor diferente para el padre y para el hijo: - Al padre se le retorna el PID del hijo. - Al hijo se le retorna 0. El valor retornado será -1 en caso de que la llamada falle y no se pueda crear un proceso hijo.

```

main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  ✓ int main(int argc, char *argv[]) {
6  ✓ printf("hello world (pid:%d)\n", (int) getpid());
7  ✓ int rc = fork();
8  if (rc < 0) {
9      // fork failed; exit
10     fprintf(stderr, "fork failed\n");
11     exit(1);
12 }
13 else if (rc == 0) {
14     // child (new process)
15     printf("hello, I am child (pid:%d)\n", (int) getpid());
16 }
17 else {
18     // parent goes down this path (original process)
19     printf("hello, I am parent of %d (pid:%d)\n",
20           rc, (int) getpid());
21 }
22 return 0;
23

```

Hijo

Padre

falla

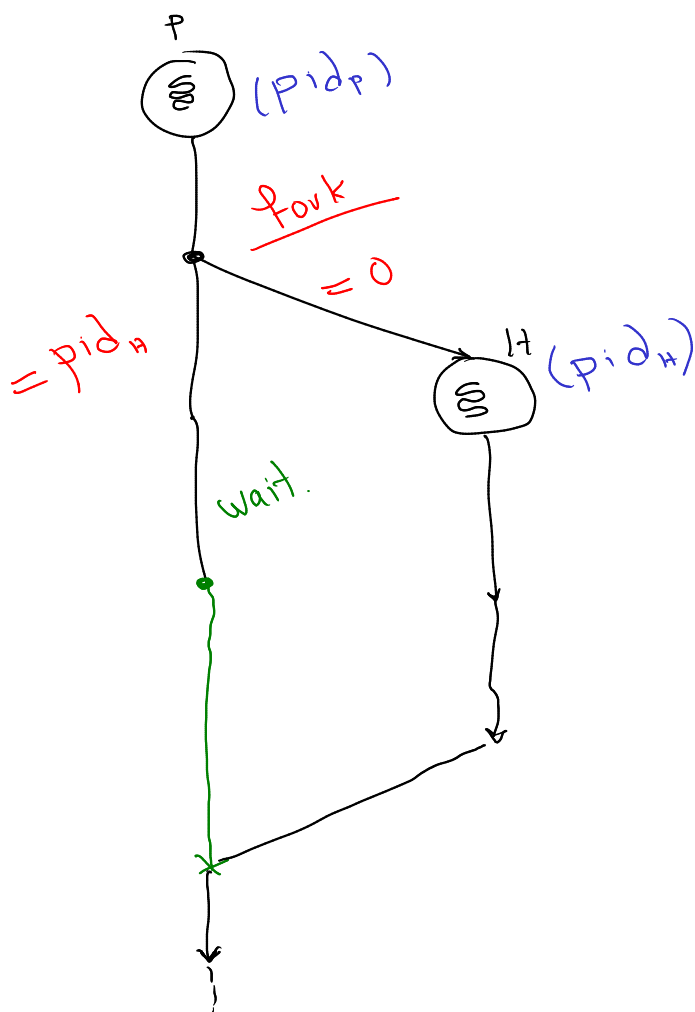
(Ambos)

```

https://p1.henryalbertoalb.repl.run
❯ clang-7 -pthread -lm -o main main.c
❯ ./main
hello world (pid:49)
hello, I am parent of 50 (pid:49) ✓
hello, I am child (pid:50) ✓

```

3. Wait. (Hace que el padre espere a que el hijo se ejecute)



wait

Permite que un proceso padre espere hasta que finalice la ejecución de un proceso hijo. El proceso padre se queda bloqueado hasta que termina el proceso hijo. Ambas llamadas permiten obtener información sobre el estado de terminación.

Función	wait
Sintaxis	pid_t wait(int *status)
Librerías necesarias	#include <sys/types.h> #include <sys/wait.h>
Descripción	Esta función suspende la ejecución del proceso padre hasta que su hijo termine.
Parámetros de entrada	status es el puntero a la dirección donde la llamada al sistema debe almacenar el estado de finalización, o valor de retorno del proceso hijo
Valor retornado	Retorna un entero que contiene el PID del proceso hijo que finalizó o -1 si no se crearon hijos o si ya no hay hijos por los cuales esperar.

③ → int main(...) {
...
exit(0);;
... }
Status

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(int argc, char *argv[]) {
7     printf("hello world (pid:%d)\n", (int) getpid());
8     int rc = fork();
9     if (rc < 0) {
10         // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     }
14     else if (rc == 0) {
15         // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         sleep(1);
18     }
19     else {
20         // parent goes down this path (original process)
21         int wc = wait(NULL);
22         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n", rc, wc, (int) getpid());
23     }
24     return 0;
25 }
```

https://p2.henryalbertoalb.repl.run

clang-7 -pthread -lm -o main main.c

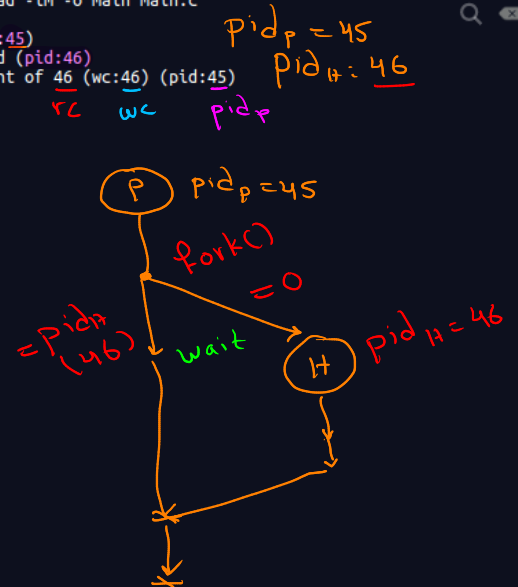
./main

hello world (pid:45)

hello, I am child (pid:46)

hello, I am parent of 46 (wc:46) (pid:45)

^



Cuando se hace un fork() padre e hijo corren la misma imagen de code.

4. exec : Cambia el programa que se ejecuta.

→ (imagen de código - ejecutable)

exe

Familia de funciones que permiten cambiar el programa que está ejecutando el proceso.

Función	Familia de funciones exec
Librerías necesarias	#include <unistd.h>
Sintaxis de las diferentes funciones	int execl(const char *path, const char *arg,...); int execlp(const char *path, const char *arg,...); int execlx(const char *path, const char *arg,...,char *const envp[]); int execv(const char *path, char *const argv[]); int execvp(const char *file, char *const argv[]);
Descripción	Esta familia de funciones, reemplaza la imagen de memoria actual del proceso con una nueva imagen de memoria. En caso de que la llamada a la función sea correcta esta no retorna nada, si hay una falla el valor retornado será -1
Parámetros de entrada	- path o file : Cadena de caracteres que contiene el nombre del nuevo programa a ejecutar con su ubicación, /bin/cp por ejemplo. - *const char arg : Lista de uno o más apuntadores a cadenas de caracteres que representan la lista de argumentos que recibirá el programa llamado. Por convención, el primer argumento deberá contener el nombre del archivo que contiene el programa ejecutado. El último elemento de la lista debe ser un apuntador a NULL. - char *const argv : Array de punteros a cadenas de caracteres que representan la lista de argumentos que recibirá el programa llamado. Por convención, el primer argumento (argv[0]) deberá tener el nombre del archivo que contiene el programa ejecutado y el último elemento deberá ser un apuntador a NULL. - char *const envp : Array de apuntadores a cadenas que contienen el entorno de ejecución (variables de entorno) que tendrá accesible el nuevo proceso. El último elemento deberá ser un apuntador a NULL.

Man exec
Ejemplos.

main.c

```

8 printf("hello world (pid:%d)\n", (int) getpid());
9 int rc = fork();
10 if (rc < 0) {
11     // fork failed; exit
12     fprintf(stderr, "fork failed\n");
13     exit(1);
14 } else if (rc == 0) {
15     // child (new process)
16     printf("hello, I am child (pid:%d)\n", (int) getpid());
17     char *myargs[3];
18     myargs[0] = strdup("wc"); // program: "wc" (word
19                             // count)
20     myargs[1] = strdup("main.c"); // argument: file to
21                             // count
22     myargs[2] = NULL; // marks end of array
23     execvp(myargs[0], myargs); // runs word count
24     printf("this shouldn't print out"); // (Esto no se muestra)
25 } else {
26     // parent goes down this path (original process)
27     int wc = wait(NULL);
28     printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
29           rc, wc, (int) getpid());
30 }
31 return 0;

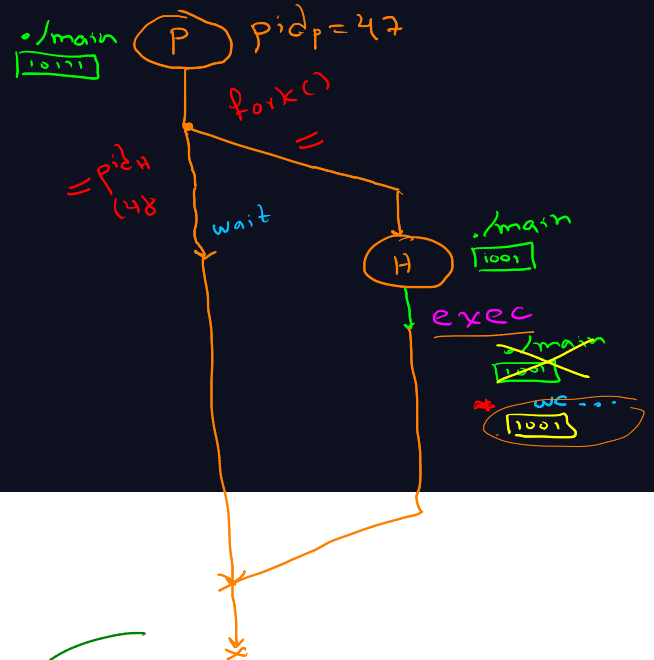
```

https://p3.henryalbertoalb.repl.run

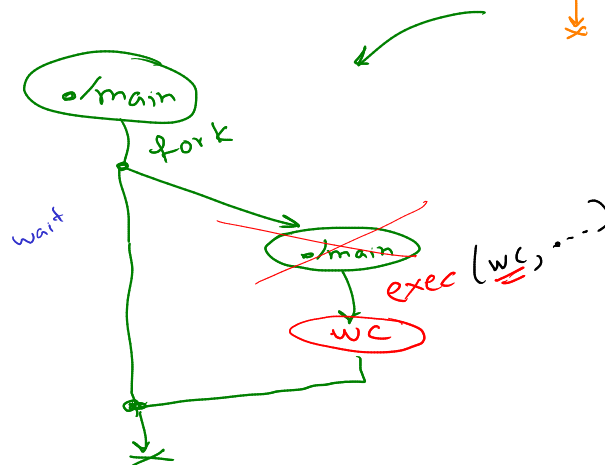
```

> clang-7 -pthread -lm -o main main.c
> ./main
hello world (pid:47)
hello, I am child (pid:48)
29 123 897 main.c
hello, I am parent of 48 (wc:48) (pid:47)

```



Hijo
* wc main.c



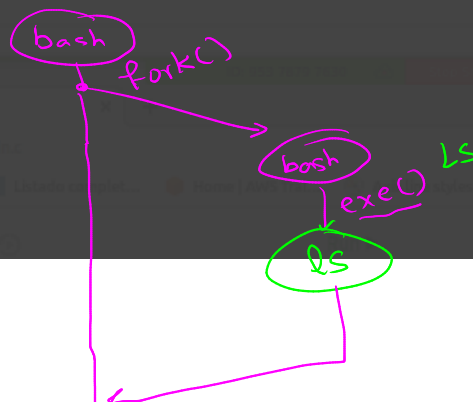
Que pasa en la consola. (bash)

File Edit View Search Terminal Help

```

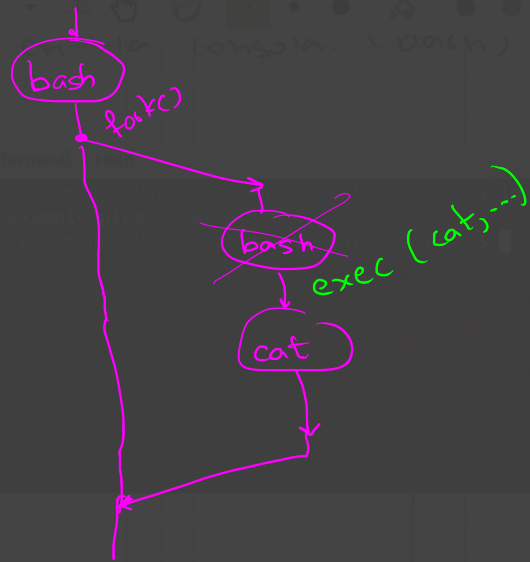
(base) tigar@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$ ls
a.out example_pipe2.c example_pipe.c practica_1
(base) tigar@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$

```



```
File Edit View Search Terminal Help
(base) tigar@fuc-pc:~/Documents/UdeA/sistemas-operativos/dudas$ cat example_pipe.c
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1

int main(void) {
    char write_msg[BUFFER_SIZE] = "Greetings\n";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;
    /* Create the pipe */
    if (pipe(fd) == -1) {
        fprintf(stderr, "Pipe failed");
        return 1;
    }
    /* fork a child process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    if (pid > 0) { /* parent process */
        /* close the unused end of the pipe */
        close(fd[READ_END]);
        /* write to the pipe */
        write(fd[WRITE_END], write_msg, strlen(write_msg)+1);
        /* close the write end of the pipe */
        close(fd[WRITE_END]);
    }
    else { /* child process */
```



→ Redirección (>) ? Como se hacen en C

```
File Edit View Search Terminal Help
(base) tigarito@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$ ls (Pantalla)
a.out example_pipe2.c example_pipe.c practica_1
(base) tigarito@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$ ls > inventario.txt
(base) tigarito@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$ ls
a.out example_pipe2.c example_pipe.c inventario.txt practica_1
(base) tigarito@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$ cat inventario.txt
a.out
example_pipe2.c
example_pipe.c
inventario.txt
practica_1
(base) tigarito@fuck-pc:~/Documents/UdeA/sistemas-operativos/dudas$
```

redirección

contenido inventario.txt

```
main.c
9 int main(int argc, char *argv[]) {
10     int rc = fork(); ① fork()
11     if (rc < 0) {
12         // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     }
16     else if (rc == 0) {
17         // child: redirect standard output to a file
18         close(STDOUT_FILENO); ② cerrar salida std
19         open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC,
20             S_IRWXU); ③ se abre archivo
21         // now exec "wc"... ④ se llama el exec()
22         char *myargs[3];
23         myargs[0] = strdup("wc"); // program: "wc" (word
24         myargs[1] = strdup("main.c"); // argument: file to
25         myargs[2] = NULL; // marks end of array
26         execvp(myargs[0], myargs); // runs word count
27     }
28     else {
29         // parent goes down this path (original process)
30         int wc = wait(NULL);
31         assert(wc >= 0);

```

```
https://p4.henryalbertoalb.repl.run
./main
ls
main main.c p4.output
cat p4.output
33 114 850 main.c
```

Processes → Jerarquías.

Shell ⇒

- C
- Archivos (lectura, ...)
- API procesos